



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-6429 netrc credential leak with reused proxy connection

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-14
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-6429 netrc credential leak with reused proxy connection - Vulnerability Report

Published: 2026-06-14 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

CVE-2026-6429 curl / libcurl netrc credential leak with reused proxy connection

1. Executive Summary

CVE-2026-6429 is a curl and libcurl information-disclosure vulnerability in HTTP redirect handling when `.netrc`-sourced credentials, clear-text HTTP, and a reused proxy connection intersect. According to the upstream curl advisory, libcurl can leak the password for the original host to the redirected-to host when an application uses a `.netrc` file for credentials, follows HTTP redirects, sends both the original request and the redirected request over clear-text `http://`, and reuses the same HTTP proxy connection. The issue is upstream-rated Medium severity and carries an NVD-enriched CVSS v3.1 base score of 5.3 / Medium.

This is not a broad internet-edge remote code execution issue, and the operational preconditions significantly narrow the attack surface. The curl project explicitly states that the flaw does not affect the curl command-line tool. Exposure is instead concentrated in applications, libraries, appliances, automation pipelines, and custom integrations that embed libcurl, use `.netrc` for host credentials, allow redirect following, and traverse a shared HTTP proxy while still permitting clear-text HTTP sessions.

The confidentiality impact can still be meaningful where those conditions exist. An attacker who controls the initial HTTP service, the redirect target, or a related intermediary workflow can potentially cause credentials intended for the first host to be sent to a second host. In enterprise environments, this matters most for service accounts, integration credentials, legacy automation, embedded software updaters, and internal tools that rely on libcurl while remaining poorly inventoried.

The immediate defensive priority is to upgrade libcurl to 8.20.0 or apply an equivalent vendor backport, then hunt for workflows that combine `.netrc`, redirect-following behavior, clear-text HTTP, and proxy-mediated traffic. Detection should emphasize software inventory, proxy and network telemetry, and configuration review rather than IOC-driven threat hunting because the reviewed public sources did not identify stable attacker infrastructure, malware payloads, or confirmed in-the-wild exploitation.

2. Intelligence Requirement

This report addresses the following intelligence requirement: what does CVE-2026-6429 mean for defenders using curl or libcurl in HTTP client workflows, which environments are genuinely exposed, how credible is exploitation, and what remediation, detection, and hunting actions should be prioritized now?

Question	Assessment
What is the flaw?	libcurl can leak <code>.netrc</code> -derived host credentials across an HTTP redirect when a reused proxy connection preserves the wrong authentication state.
What must happen first?	<code>.netrc</code> credentials must be in use, redirects must be followed, both the original and redirected requests must use clear-text <code>http://</code> , the same HTTP proxy must be used, and the proxy connection must be reused.
Which product is affected?	curl/libcurl library behavior; the upstream advisory says the curl command-line tool itself is not affected.
What is the primary impact?	Confidentiality loss: a password intended for the first host can be disclosed to the followed-to host.
Which versions are affected?	curl/libcurl 7.14.0 through 8.19.0 according to the upstream advisory and vendor JSON metadata.
What fixes the issue?	Upgrade to curl/libcurl 8.20.0 or apply a downstream vendor

	backport containing the upstream fix commit.
Is exploitation confirmed?	No authoritative evidence of in-the-wild exploitation was identified during this review, and the CVE was not present in the CISA KEV catalog at review time.
Which environments deserve highest priority?	Applications embedding libcurl behind explicit HTTP proxies, especially legacy automation and service-account workflows that still permit clear-text HTTP and rely on `.netrc` credentials.

3. Source Review and Confidence

Source	Contribution	Confidence
curl security advisory (`CVE-2026-6429.html`)	Canonical flaw description, trigger conditions, affected versions, fixed version, and mitigation guidance	High
curl machine-readable advisory JSON (`CVE-2026-6429.json`)	Machine-readable version ranges, fix and introduction commits, CWE mapping, publication date, and advisory metadata	High
NVD CVE record	CVSS v3.1 vector and severity enrichment for defender prioritization	High
Debian Security Tracker	Downstream package status and evidence that several distro branches still tracked vulnerable curl packages during review	High
Ubuntu CVE / USN pages	Additional downstream packaging and severity context, including fixed package versions in supported Ubuntu releases	High
CISA KEV feed and FIRST EPSS feed	Exploitation-priority context showing no KEV presence and a low EPSS probability at review time	High
Microsoft Security Update Guide intake item	Useful as syndication context and workflow provenance only; not the primary technical source	Medium

Assessment confidence is high for the core technical findings because the upstream curl advisory, vendor JSON, NVD record, and multiple downstream package trackers all align on the same trigger sequence and fixed-version boundary. Confidence is medium for exploitation prevalence because the reviewed public record did not include confirmed campaign use, incident-response reporting, or curated victimology tied to this CVE.

Three caveats are important for defenders. First, the flaw requires a precise combination of conditions and should not be treated like a generic "all curl traffic is unsafe" issue. Second, the upstream advisory explicitly distinguishes libcurl from the curl command-line tool, which reduces false positives during triage. Third, distro package owners may backport the upstream fix without matching the upstream version string exactly, so software inventory results should always be reconciled against vendor package advisories before exposure is declared.

4.1 Vulnerability metadata

Field	Value
CVE	CVE-2026-6429

Product family	curl and libcurl
Affected component	libcurl HTTP redirect / auth handling over reused proxy connections
Weakness	CWE-200 Exposure of Sensitive Information to an Unauthorized Actor
Vendor severity	Medium
NVD severity	CVSS v3.1 5.3 / Medium
CVSS vector	`AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:N/A:N`
Affected versions	curl/libcurl 7.14.0 through 8.19.0
Fixed version	curl/libcurl 8.20.0
Introduced by	curl commit `01165e08e0d131b399fba2190f17af67e66f0888`
Fixed by	curl commit `b4024bf808bd558026fdc6096e8457f199ace306`
Primary impact	Confidentiality loss through credential disclosure to a redirected host
Upstream publication date	2026-04-29
Exploitation status	No authoritative in-the-wild exploitation confirmation identified during this review
EPSS at review time	0.00021 (6.07th percentile) on 2026-06-13

4.2 Flaw mechanics

The bug sits in redirect and credential-reset logic, not in cryptography or proxy authentication itself. The upstream patch shows curl changed how it clears credentials on redirect, with explicit test coverage added for a `.netrc` plus redirect-plus-proxy scenario. In practical terms, libcurl could preserve `.netrc`-derived authentication state too long when the redirect target was reached over the same reused HTTP proxy connection, allowing a password associated with the first host to be sent to the redirected host.

The advisory defines the preconditions narrowly: the original URL and the redirect URL must both be clear-text `http://`; the client must be configured to use a `.netrc` file for credentials; redirects must be followed; the same HTTP proxy must be used; and the same connection must be reused. This is therefore a workflow-level exposure problem, not a general failure of TLS or a universal credential leak on every redirect.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
Applications embedding libcurl behind a shared or explicit HTTP proxy	Directly matches the vendor-described trigger conditions	Immediate
Legacy automation using `.netrc` for service credentials and following redirects	Creates a practical path for host-to-host credential confusion	Immediate
Internal tools or appliances still using clear-text `http://` for compatibility	Supplies the insecure transport precondition needed for the bug to matter	High

Environments with weak software bill-of-materials visibility for bundled libcurl	Makes vulnerable library usage easy to miss during asset triage	High
Systems patched by downstream vendors with backports	Require careful package-level validation rather than upstream version-only checks	Medium
Pure curl CLI usage without affected libcurl embedding	Reduced priority because the upstream advisory says the curl tool is not affected	Reduced

4.4 Downstream package observations

Source	Observation	Defensive meaning
Ubuntu USN-8227-1	Fixed package versions include `8.5.0-2ubuntu10.9` for Ubuntu 24.04 LTS and `7.81.0-1ubuntu1.24` for Ubuntu 22.04 LTS	Version-only checks must account for distro backports
Debian Security Tracker	bullseye, bookworm, and trixie entries were still marked vulnerable at review time, while unstable had a fixed line at `8.20.0~rc3-1` / `8.20.0-5` in sid/forky context	Exposure can persist in maintained distro branches even when upstream has already released a fix
curl upstream advisory	Recommends upgrading to 8.20.0, applying the patch, or avoiding the combination of `.netrc`, HTTP, and HTTP proxy usage	Configuration workarounds are viable if patching is temporarily delayed

4.5 Defensive meaning and ATT&CK mapping

ATT&CK mapping should remain conservative because CVE-2026-6429 is a conditional credential-exposure flaw rather than an actor tradecraft set. Where mapping is useful, it primarily describes the protocol surface and the type of confidentiality loss defenders should consider.

ATT&CK ID	Technique	Rationale
T1071.001	Application Layer Protocol: Web Protocols	The vulnerable operational path is clear-text HTTP traffic with redirect following through a proxy.
T1040	Network Sniffing	If exposed workflows traverse observable networks, leaked credentials sent over clear-text HTTP can be collected by on-path observers.

Higher-confidence ATT&CK mapping beyond the above is not strongly supportable from the reviewed public evidence. The published sources describe a software defect and remediation path, not a confirmed adversary playbook.

5. Threat Context

CVE-2026-6429 is best understood as an exposure-management and secure-integration problem. The flaw is unlikely to produce indiscriminate mass exploitation because it requires a fairly specific and uncommon combination of `.netrc` credential use, redirect following, clear-text HTTP, proxy reuse, and vulnerable libcurl behavior. That constraint lowers urgency relative to pre-authentication RCEs, but it does not eliminate enterprise risk where credentials are long-lived, service-scoped, or reused across sensitive automation.

The most relevant threat scenario is not a random internet-wide scan. Instead, risk clusters around legacy internal tools, integration middleware, package fetchers, update mechanisms, and business workflows where libcurl is embedded behind the

scenes and defenders may not realize the dependency exists. If such a workflow talks to an attacker-controlled or attacker-influenced HTTP endpoint that issues a redirect, the attacker may receive credentials meant for the original host.

5.1 Representative attack and failure scenarios

Scenario	Description	Defensive meaning
Malicious redirect target	An attacker controls or compromises an HTTP endpoint that causes a vulnerable libcurl client to follow a redirect to an attacker-visible host	Review redirect behavior and remove clear-text HTTP dependencies
Legacy proxy-mediated automation	A service account uses <code>.netrc</code> through libcurl behind an enterprise proxy while following redirects during update, sync, or retrieval tasks	Prioritize service-account and scheduled-task workflows for validation
Shared or partner network traversal	Clear-text proxy-mediated HTTP requests traverse infrastructure that is not fully trusted or tightly segmented	Treat exposure as both credential-disclosure and transport-governance risk
Hidden embedded dependency	A vendor appliance or business application bundles libcurl but asset owners only track the top-level application version	Use SBOM, package inventory, and vendor advisories to avoid blind spots

5.2 Exploitation-status assessment

No authoritative evidence of active exploitation was identified in the reviewed upstream curl material, NVD record, Debian/Ubuntu downstream references, or the CISA KEV feed. FIRST EPSS data reviewed on 2026-06-13 also indicated a low predicted exploitation probability (`0.00021`, 6.07th percentile). That evidence supports a moderate, condition-dependent prioritization posture rather than emergency incident-response assumptions.

That said, low observed exploitation is not the same as low consequence for every environment. Organizations with brittle legacy automation, hard-coded redirect-following workflows, or proxies spanning trust boundaries can still face meaningful service-account credential disclosure if the affected libcurl path exists.

5.3 Exposure-reduction priorities

Exposure driver	Why it increases risk	Defensive priority
Use of <code>.netrc</code> for noninteractive service credentials	Supplies the credential source implicated by the advisory	Highest
Allowing clear-text <code>http://</code> for production automation	Preserves the transport precondition required for the flaw	Highest
Redirect following to untrusted or external destinations	Creates the host transition where credential leakage can occur	Highest
Explicit HTTP proxy use with long-lived connection reuse	Matches the vendor-described reused proxy connection condition	High
Limited dependency visibility for embedded libcurl	Keeps vulnerable code paths hidden during remediation	High
Vendor-backported packages not represented in upstream version strings	Can distort vulnerability reporting if triage relies only on version regexes	Medium

6. Detection and Hunting

This CVE is better handled through exposure discovery and behavioral review than through classic IOC matching. No malware hashes, domains, or campaign-linked infrastructure were identified in the reviewed source set. Detection should therefore focus on finding vulnerable library versions, identifying clear-text HTTP workflows that traverse proxies, and surfacing proxy or network telemetry that suggests `Authorization` headers or service-account traffic may be flowing to unexpected destinations.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Coverage caveat
`CVE-2026-6429` present in vulnerability inventory	EDR TVM / vulnerability-management tables	Directly locate endpoints already normalized as affected	Depends on platform feed freshness
Installed `curl` / `libcurl` package versions	Software inventory, package management, SBOM data	Identify potentially exposed hosts and embedded dependencies	Requires backport-aware interpretation
Non-browser processes using clear-text HTTP and common proxy ports	EDR network events, proxy logs, firewall logs	Surface candidates for vulnerable libcurl workflows	Not a proof of exploitation
Proxy or web logs showing clear-text HTTP requests with `Authorization` headers from `curl` / `libcurl` user agents	Proxy logs, secure web gateway, packet capture	Detect higher-risk credential-exposure patterns	Many environments do not log headers at this level
Redirect chains from one host to another in legacy automation	Proxy logs, application logs, HTTP tracing	Identify business workflows that need review even without confirmed compromise	Often requires custom log correlation

6.2 KQL - direct vulnerability lookup

```
DeviceTvmSoftwareVulnerabilities
| where CveId == "CVE-2026-6429"
| project DeviceName, SoftwareName, SoftwareVersion, VulnerabilitySeverityLevel, CveId
| order by DeviceName asc
```

6.3 KQL - proxy and clear-text HTTP exposure hunt

```
let lookback = 30d;
let proxyPorts = dynamic([8080, 3128, 8000, 8888]);
DeviceNetworkEvents
| where Timestamp > ago(lookback)
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe", "firefox.exe", "opera.exe", "safari.exe")
| where RemotePort == 80 or RemotePort in (proxyPorts)
| summarize FirstSeen=min(Timestamp), LastSeen=max(Timestamp),
    RemoteIPs=make_set(RemoteIP, 10), RemoteUrls=make_set(RemoteUrl, 10),
    Ports=make_set(RemotePort, 10), SampleCommands=make_set(InitiatingProcessCommandLine, 10)
    by DeviceName, InitiatingProcessFileName
| order by LastSeen desc
```

6.4 Sigma - clear-text authorization over proxy logs

```
title: Potential Cleartext HTTP Authorization Header From curl or libcurl User Agent
id: 7f3f6f8d-9914-4a4f-a80c-7e9dd25eb503
status: experimental
description: Flags proxy-log events where a curl or libcurl-style user agent issues a clear-text HTTP request that includes an Authorization header. This is an exposure-review analytic relevant to CVE-2026-6429, not a standalone exploitation detector.
author: Lost Boys Cyber
```



```

date: 2026-06-14
references:
  - https://curl.se/docs/CVE-2026-6429.html
  - https://nvd.nist.gov/vuln/detail/CVE-2026-6429
logsource:
  category: proxy
detection:
  cleartext_url:
    url|startswith: 'http://'
  curl_ua:
    user_agent|contains:
      - 'curl/'
      - 'libcurl/'
  auth_header:
    request_headers|contains:
      - 'Authorization: Basic '
      - 'Authorization: Digest '
  condition: cleartext_url and curl_ua and auth_header
falsepositives:
  - Legitimate legacy automation over HTTP
  - Security testing and protocol validation
  - Proxy products that reconstruct or normalize headers unexpectedly
level: medium
tags:
  - attack.t1071.001
  - attack.t1040

```

6.5 Threat-hunting hypotheses

Hypothesis	What to prove or disprove
Some legacy applications still use clear-text HTTP behind enterprise proxies while relying on embedded libcurl	Correlate software inventory with network events on port 80 and common proxy ports
Service accounts using `.netrc` in unattended jobs remain present in production	Review automation hosts, deployment manifests, home directories, and secret-management exceptions
Redirect-following business workflows can transition between trusted and untrusted hosts	Inspect proxy logs and application traces for cross-host HTTP redirect chains
Vulnerable code is hidden in bundled third-party software rather than visible `curl` packages	Compare package inventory, SBOMs, container layers, and vendor appliance advisories

Threat hunting should be framed as workflow validation, not as a search for attacker-controlled IOCs. If defenders can eliminate clear-text HTTP and `.netrc`-driven service credentials from libcurl-dependent workflows, the practical risk of this CVE drops sharply even before every endpoint has been fully re-inventoried.

7. Recommendations

Priority	Owner	Action	Rationale
Immediate	Platform / endpoint engineering	Upgrade curl/libcurl to 8.20.0 or deploy vendor backports that include the upstream fix for CVE-2026-6429	Removes the vulnerable redirect credential-handling behavior
Immediate	Application owners	Identify applications, appliances, and integrations that embed libcurl and use `.netrc` for noninteractive credentials	The vulnerable surface is often hidden behind higher-level software
Immediate	Network / proxy engineering	Eliminate or tightly restrict	Removes a required

		clear-text <code>`http://`</code> for automation and service-account workflows	precondition for the flaw
Immediate	DevOps / integration owners	Audit redirect-following logic and disable unnecessary redirect chasing for credentialed HTTP clients	Reduces the host-transition condition needed for leakage
High	Security engineering	Add software-inventory and network-hunt coverage for curl/libcurl plus clear-text proxy-mediated HTTP activity	Best available way to locate realistic exposure
High	IAM / secrets management	Migrate service-account authentication away from <code>`netrc`</code> where practical and rotate credentials after patching high-risk workflows	Shrinks the credential source and limits blast radius
Medium	Vulnerability management	Reconcile upstream versions with distro backports from Ubuntu, Debian, and other package maintainers before declaring hosts vulnerable or safe	Prevents false positives and false negatives
Medium	QA / engineering	Add regression tests for credential clearing across cross-host redirects and proxy reuse in libcurl-dependent software	Prevents the same class of auth-state persistence issue from reappearing unnoticed
Low	Architecture	Prefer TLS-only, redirect-minimized, explicitly authenticated service integrations over legacy HTTP plus <code>`netrc`</code> patterns	Reduces structural reliance on risky legacy workflows

Recommended execution order:

- 1. Pull vulnerability-management and software-inventory reports for curl/libcurl exposure.
- 2. Patch or backport affected libcurl instances, prioritizing systems that use proxies and noninteractive credentials.
- 3. Remove or block clear-text HTTP from production automation wherever business constraints allow.
- 4. Review redirect-following integrations and rotate any credentials associated with high-risk workflows after remediation.
- 5. Hunt for residual proxy-mediated clear-text HTTP behavior and undocumented embedded libcurl dependencies.

8. References

- 1. curl Security Advisory, ``CVE-2026-6429: netrc credential leak with reused proxy connection`` - <https://curl.se/docs/CVE-2026-6429.html>
- 2. curl machine-readable advisory JSON, ``CVE-2026-6429.json`` - <https://curl.se/docs/CVE-2026-6429.json>
- 3. NVD CVE record, ``CVE-2026-6429`` - <https://nvd.nist.gov/vuln/detail/CVE-2026-6429>
- 4. Debian Security Tracker, ``CVE-2026-6429`` - <https://security-tracker.debian.org/tracker/CVE-2026-6429>
- 5. Ubuntu CVE page, ``CVE-2026-6429`` - <https://ubuntu.com/security/CVE-2026-6429>
- 6. Ubuntu Security Notice, ``USN-8227-1: curl vulnerabilities`` - <https://ubuntu.com/security/notices/USN-8227-1>

- 7. CISA Known Exploited Vulnerabilities Catalog feed - https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json
- 8. FIRST EPSS API record for `CVE-2026-6429` - <https://api.first.org/data/v1/epss?cve=CVE-2026-6429>
- 9. Microsoft Security Update Guide syndication entry - <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-6429>